

derpy's script server

written by Snow, a.k.a. derpy54320

May 23rd, 2026

Thanks for downloading derpy's script server (DSS). This is custom server software used to host a custom server for Bully: Scholarship Edition (PC), that people can connect to using derpy's script loader (DSL). This manual is meant as a brief guide to help you get your server setup, and to help you understand a few things you should refrain from.

Some legal notes first.

I'd just like to take a quick moment to say once more as clearly as possible, **derpy's script server (DSS) is custom server software not in any way affiliated with or endorsed by Take-Two Interactive or Rockstar Games**. Similarly, **derpy's script loader (DSL) is a third party modification used to connect to servers, and is also not affiliated or endorsed**.

Rockstar has a short article on their guidelines for [GTA RP](#). Again, the existence of that article does **not at all constitute endorsement**. They do state that they generally de-prioritize legal action against mods and servers that follow those guidelines though, so familiarize yourself with them and respect them to the best of your ability.

In the past there's been a fairly high profile custom server that used DSS that was shut down, likely because it violated several of the guidelines in that GTA RP article. Although DSS itself was not targeted, you should treat the shutdown of that old server as a cautionary tale and **never use game names or assets in the branding of your server**, and also **never try to generate revenue for your server through a support page**.

Creating your server folder.

DSS was designed to run on both Windows and Linux, though it is primarily tested on Linux and that is my official suggestion for it. The Windows version is better suited for local testing than actual server hosting in my opinion.

Whichever you pick, you should **create a folder for your server**. For the sake of the rest of this manual, let's say we created one called **my_server**.

- If you're using **Windows**, put **derpy_script_server.exe** in **my_server**.
- If you're using **Linux**, put **derpy_script_server** in **my_server**.

Next we'll learn how to actually run the server. This will depend on your operating system, so I'll split it into two sections.

Running the Windows server.

If you're using Windows, simply double click **derpy_script_server.exe** to run it. It will generate a **default dslconfig.ini** the first time it runs, then exit. Do not run the server again yet, as we'll want to configure it first.

Running the Linux server.

Although DSS should work on many distributions, I've only tested it on Ubuntu and Debian. If you're just now deciding which operating system to use, I suggest **Ubuntu**. To run the server you'll need to do two things first: **add execute permissions** to the file, and if you don't have it already **install the 32 bit version of libc6** since DSS is currently 32 bit.

To add execute permissions to the file, run **chmod +x**. Installing the 32 bit libraries could vary by distro, but if you have **apt** you can run **sudo apt install libc6-i386**.

Once you run the server for the first time, it will generate a **default dslconfig.ini**. Do not run the server again yet, as we'll want to configure it first.

Configuring your dslconfig.ini.

By now, you should have a default **dslconfig.ini** file in **my_server** (assuming you ran the server in its own folder). Let's edit this before we run the server again.

The **[SERVER]** section should be self-explanatory, so you may edit it as needed. You'll also notice there's a **read_eula** option in there, which you'll need to turn on. If you did not already read the **eula.txt** included in the server release, you should do so now.

The **[ACCOUNT_CONFIG]** section can get pretty complex, and is something you shouldn't worry about until we get to the **Setting up accounts** section.

All other sections are usually fine by default. It's very rare that DSS changes its config version, you should back your config up when switching DSS versions in case it updates it.

Allowing your port to be accessed.

Now that we're all set up, we're almost ready to run the server for real. Just need to make sure the port you specified in **server_port** is accessible. By default it's **17017** (and usually shouldn't be changed), so you'll want to allow that in your firewall and port forward if necessary. The specifics on doing that is outside the scope of this manual.

Running your server and connecting.

Now you're ready to run your server. You can run it the same way you did earlier to just get it running, and allow yourself to type commands into the console. You can use **/help** to get a list of commands (the **/** is always optional, and mostly just conveys that it's a command).

To connect to your server, you'll need to download derpy's script loader (DSL). The latest version is available on Nexus Mods. Once you've got that installed, you can connect to the server using a command line option. You can edit the launch options for the game on both Steam and the Rockstar Games Launcher, **or** you can put a file called **commandline.txt** in your game's root directory. This file is read by DSL to append to the internal command line.

To connect to a server, use **--joinServerASAP ip:port** where **ip** is the hostname you want to connect to, and **port** the port. If using the default port **17017**, you can skip the **:port** part.

This option tells DSL to connect to a server as soon as the game starts, **and** to put the game into an "empty" state so the server can have better control of the game. Most server owners decide to require this method of joining, although by default you are also allowed to connect to the server by simply using the **/connect** command in-game.

While we're on the subject, just a quick note. You can also use **--username user** to specify a username when joining a server, and **--password pass** to specify a password if desired. It's usually suggested to not use the password option for security, though it's a valid option if you're sure that you're the only one on your PC.

Installing some mods.

You'll have probably noticed by now that playing on the server doesn't feel very interesting. This is because you don't have any mods installed yet, and DSS is designed to leave all the interesting stuff to mods. Finding and downloading mods is entirely up to you, but when you do find some you'll need to put them in a **scripts** folder. If you don't have that folder yet, make an empty one.

The **scripts** folder should consist mostly of folders for mods. Each of those folders is considered its own "mod", and will usually have a **config.ini** file directly inside it. Since you'll likely get many mods eventually, it's suggested you also make use of **category folders**. To make a new category folder, just make a new folder with square brackets around it.

For example, **[THIS_IS_A_CATEGORY]**. You can put your mods into these category folders to help keep things organized. If desired, you're also allowed to nest categories.

Understanding the built-in sync.

This may be a bit of a complicated read, but you should understand the core concepts related to how the built-in sync works before we talk about how to set it up.

DSS and DSL have optional built-in sync. It's on by default in your DSS config, and basically means the server is responsible for the creation of peds and vehicles (collectively referred to as **entities**). When a client (DSL) connects to a server with this built-in sync enabled, the way entities are handled is completely different. It will try to both delete and hide non-synced entities from you, and manage the spawning of synced entities.

Technically, these **entities are just data managed by the server**. The client then attempts to represent them within its hard-coded limits. The game can only safely spawn about 20 peds, so even if you have 100s on the server you'll only see the nearest few on the client.

The **server does not “run” the game** in any way. Instead, **entities are assigned an “owner”**. In general, an entity is owned by the nearest player. This player tells the server about changes to the entity's state, which the server will validate and apply. The server still always has authority over the final state of an entity, and will reject updates from the owner until the owner is “caught up” to the state the server wants for that specific data field.

This built-in sync is **modular and intentionally minimalistic**. Only the **model, position, area, ped's vehicle, and vehicle's peds** are synced by default. New data fields can be scripted though, so it's up to mods to provide more in-depth sync. The built-in sync just worries about the core state and logic. **You'll likely download a mod to do the extra sync.**

I know this is a lot to take in, but the last thing you should really understand is the concept of **dimensions**. Every entity is stored in a dimension, which is essentially like an “instance”. By default there is only one dimension (the “**main dimension**”). Other dimensions are only used if a mod scripts it, for something like an isolated environment (like an interior or minigame for example). There can be **512 entities per dimension**, and **128 total dimensions**. Players only see entities that are in the same dimension, and other entities are **inactive**.

Using the built-in sync.

Now that you understand how the built-in sync works, you can choose if you want it enabled. If not, you can disable **use_sync** in your server config (since it's on by default).

Like we discussed, sync is very limited by default. You'll just be floating around for other players unless you get a mod designed to improve the sync. Luckily there is such a mod by yours truly called **Syn+**. It's suggested that virtually every server uses this if it wants sync. Currently it's available on Nexus Mods, though it could be moved in the future.

Setting up accounts.

This is usually the biggest barrier to entry for new server owners as it's the most technical, but I'll do my best to break everything down. If you want to use the account system provided by DSS then you'll need to do quite a few things. First I'll provide an overview, then we'll go over each of these steps in-depth.

- Enable accounts by setting up SSL in `dslconfig.ini`.
- Enable the account registration API in `dslconfig.ini`.
- Host a website that can trigger account registration.
- Make the web server pass account registration to the API.
- Define some custom roles for accounts if needed by mods.

If accounts are not enabled, then users cannot sign in with a password. If accounts are enabled and you'd like to disable guests, make sure to turn on **account_only** in your config.

Setting up SSL.

Accounts are enabled by setting up SSL in your config. There is no support for insecure accounts, and account data is only ever sent over secure connections. The server switches from a secure connection to normal after the sign-in process by default for performance.

Anyway, yes you'll need to get yourself an SSL certificate for this. On Linux my personal recommendation is **acme.sh**, but either way the specifics of getting an SSL cert are outside the scope of this manual. It should be easy enough to find other guides not related to DSS.

Once you have your SSL cert, go into your `dslconfig.ini` and set **ssl_key** to a path to your private key. Then you'll use **ssl_chain** if using a "full chain certificate", or use **ssl_cert** if using a single certificate. Do not use both. All files are expected to be PEM files.

Once you've done this and the server launches without any errors, accounts are enabled. You can manage accounts using the `/account` server command (use `/help accounts`). Before moving on, it is a good idea to create a test account through the server to make sure that your SSL configuration is working correctly when you connect with that account.

Setting up account registration.

If you want users to be able to create accounts on their own, you need to set up the basic web API. This sounds more difficult than it is, just uncomment **signup_ip** and **signup_port** in your server config. This tells DSS to run a simple web API for account registration.

Setting up your registration page.

By this point, you should have accounts working fully for your server. You've also set up the web API provided by DSS (we'll call it the **account API**). Now it's time to make a user interface for account creation, and in this example we'll do it using a website.

I won't go over the play-by-play on how to set up a website, that's something you'll need to familiarize yourself with first. If you're using Linux, my personal recommendation is **nginx**. The website you set up is what should be visible to the public, while the account API is only available to the server itself. This is why the default value for **signup_ip** is **127.0.0.1**, it refers specifically to the server itself (the only IP that can use the API).

How do we actually make a webpage that can use that API? Well, we need to give the user a way to send a **POST request** to the account API. The simplest way to do this is through an **html form**, though later we'll talk about using JavaScript to do it a little better.

The only action that the account API currently supports is **/signup**. It requires 3 values to be given to it using the **x-www-form-urlencoded** format (the default for html forms). Those values should be **"username"**, **"password"**, and **"confirm"** (the password again).

Here's an **example html form** that can **POST /signup** with the needed values.

```
<form action="/signup" method="post">
  <label for="user">Username:</label>
  <input id="user" name="username" type="text"/>
  <br/>
  <label for="pass">Password:</label>
  <input id="pass" name="password" type="password"/>
  <br/>
  <label for="check">Confirm:</label>
  <input id="check" name="confirm" type="password"/>
  <br/>
  <input type="submit"/>
</form>
```

The **name** attributes of each input are what gets used when sending the data, so be sure to keep those. The method is **POST** like we said, so it'll send a **POST request** when you submit.

Quick problem though... see **/signup**? It has no way of knowing that it's meant to be interacting with the account API. It's going to just try and get that from your website and probably find nothing. What we need to do is set up a **proxy pass** in your web server config. We want to "pass" any **/signup** to our account API.

Setting up a proxy pass to DSS.

Your web server should pass account creation requests to the account API we set up earlier. The way you configure this will depend greatly on what web server you are using, but for **nginx** you can do it using the **proxy_pass** directive along with a few others. Here's an example of how to configure it in **nginx**.

```
location = /signup {  
    proxy_http_version 1.1;  
    proxy_pass https://127.0.0.1:17018/signup;  
    proxy_set_header X-Real-IP $remote_addr;  
}
```

We set the HTTP version for this pass to **HTTP/1.1** because the account API only supports that version. The address we used in our pass is the one we configured the account API for earlier, and the **X-Real-IP** header is used to pass the IP of the user who tried to create the account. If **X-Real-IP** is given, DSS will use it to uphold IP bans and limit total accounts.

In **nginx**, the `=` sign means the requested location must match the string on the right exactly. Without that sign, it would use the proxy pass for any location simply starting with that string. You may notice that in this example, I pass **/signup** to **/signup**. The reason it passes to **/signup** is because that's the **POST** action expected by DSS, but you are of course totally free to change the location used to trigger the pass. For example, you could make the **location = /anything/for/example** as long as you use the same path in your html form.

Anyway, now whatever response the account API gives is going to be given to the user directly. DSS tries to be lightweight with its API, so this response is not pretty by any means. You can configure your web server to intercept error codes to make those look prettier, but the **200 OK** status it returns when your account is created will still be plain. For this reason, it's suggested you use JavaScript for the request instead of a standard form.

Response codes.

If you decide to use JavaScript for your signup request, you should know the error codes that DSS can return for **/signup**. There's only 4, so they'll be easy to cover.

- **200 OK:** your account was created successfully. The body of this response is plain text informing you that your account was created.
- **400 Bad Request:** the request was considered bad for some reason. This happens if your request isn't POST, a known action, or some expected values are missing.
- **403 Forbidden:** the request was valid, but the user is not allowed to create an account. The body of this response is plain text explaining why.
- **500 Internal Server Error:** the account couldn't be created due to an error.

Defining custom account roles.

All accounts can have up to 32 roles. This limit is imposed by the very optimized way accounts are stored, where roles are stored in a 4 byte bitfield for each account.

These **roles don't do anything on their own**, they're just a mechanism for mods to know which players should have permissions to do certain things.

Anyway, the actual names of these roles come from your server config. You'll find that by default there is only one, defined in the "**account_role admin**" line. You can add more roles by simply copying this line to more lines, and switching the role name from "**admin**" to whatever you want. Doing this enables the role to be given to accounts using the **/account** command, or by using some script functions. The order of roles is saved alongside account data, so you don't have to worry about changing the order in your config. If however you remove one and start the server, the missing role(s) will be stripped from all accounts.

Well designed mods usually allow themselves to be configured to require different role names for different permissions, though expecting there to be an "**admin**" role is standard.

The names of roles are completely arbitrary and up to the community, but some commonly used ones by me include the following. Again though, these are only here as examples. They don't do anything on their own unless you have a mod configured to use them.

- "**admin**": the built-in default admin role, allows most admin functionality.
- "**superadmin**": for super restricted stuff, like triggering server commands.
- "**moderator**": for moderate stuff, like tools to warn / kick players.
- "**banned**": for account bans, telling a server script to deny connections.

Thanks for reading.

Thanks for reading, and thanks for taking interest in setting up your own server. I hope this guide helped you get things started, and I hope you can find some good mods to make the server your own. Who knows, maybe you're even interested in making your own?

Good luck and have fun! <3